EA2. Creación de Repositorio:
Procesamiento Funcional de Datos para el Sistema de Gestión de Transporte Urbano TecnoMovil Data

Estudiante:
Carlos Alberto Millán Castaño

Grupo:
POII-ACTIVIDADES 8

Asignatura: Programación Orientada a Objetos II (PREICA2602B020046)
Docente: Ing. Jorge Armando Julio Cruz
Facultad de Ingeniería y Ciencias Básicas
Institución Universitaria Digital de Antioquia
6 de noviembre de 2026

Introducción

En el desarrollo de software contemporáneo, la articulación de la Programación Orientada a Objetos (POO) con el paradigma de la Programación Funcional (PF) y los sistemas distribuidos de control de versiones constituye un pilar para construir aplicaciones declarativas, mantenibles y menos propensas a errores de estado. Como señalan Field y Harrison (1988), la programación funcional concibe la computación como la evaluación de funciones matemáticas evitando de forma explícita el cambio de estado y el uso de datos mutables. Esta estructura conceptual, fundamentada en el cálculo lambda propuesto por Church (1941), promueve la inmutabilidad y la ausencia de efectos colaterales, lo que favorece la construcción de código predecible, fiable y fácil de razonar (Hughes, 1989). En consecuencia, la reducción de efectos secundarios resulta determinante en contextos donde la paralelización eficiente, la prevención de condiciones de carrera y el análisis de datos a gran escala son críticos (Felleisen et al., 2017; Lattner & Zeller, 2012).

Esta necesidad se refleja en el escenario de modernización del transporte urbano de la ciudad de TecnoValle a través del proyecto *TecnoMovil Data*, donde diariamente se movilizan más de 450.000 pasajeros y se generan millones de registros desde validadores electrónicos, sensores de estación, dispositivos IoT y aplicaciones móviles. El sistema anterior, construido bajo un enfoque imperativo secuencial y altamente acoplado, presentaba retrasos de hasta dos horas diarias, costos elevados de mantenimiento y errores en los conteos producto de condiciones de carrera sobre estructuras compartidas mutables. Para dar solución a este desafío, el presente informe documenta el modelado en UML (Fowler, 2004; Object Management Group [OMG], 2017), la implementación funcional en Java estándar sin frameworks externos (Groussard, 2014; Deitel & Deitel, 2018; Oracle, 2024) y la gestión del repositorio en Git y GitHub aplicando el flujo de ramificación entre la rama de desarrollo (`develop`) y la rama principal (`main`) (Chacon & Straub, 2014; Loeliger & Verburg, 2012; GitHub, 2024).

Objetivos

Objetivo General

Desarrollar un módulo funcional y declarativo en Java estándar para el sistema de gestión de transporte urbano *TecnoMovil Data*, aplicando funciones puras, inmutabilidad, expresiones lambda, funciones de orden superior y flujos (`Streams`) en conjunto con el modelado UML y los principios de diseño limpio (Fowler, 2004; Martin, 2008; Hutton, 2016), alojando la solución y su historial de versiones en la rama principal (`main`) de un repositorio en GitHub (Chacon & Straub, 2014; GitHub, 2024).

Objetivos Específicos

- Modelar la estructura estática del sistema mediante un diagrama de clases UML (OMG, 2017; Fowler, 2004; Larman, 2004) aplicando criterios de alta cohesión, bajo acoplamiento y los principios SOLID (Martin, 2003, 2008), separando la lógica de análisis de la capa de presentación en consola (Fowler, 2002; Sommerville, 2016).
- Representar los registros de validación de pasajeros y los objetos de resultado mediante estructuras de datos estrictamente inmutables (`record` y colecciones no modificables) para eliminar los efectos colaterales y el riesgo asociado al estado mutable (Field & Harrison, 1988; Hughes, 1989; Lattner & Zeller, 2012).
- Implementar de manera declarativa mediante la API de `Stream`, expresiones lambda (Church, 1941; Oracle, 2024; Groussard, 2014) y funciones puras los seis requerimientos funcionales del caso de estudio: afluencia por estación, identificación de horas pico, rutas más utilizadas, patrones de viaje por usuario, tiempo promedio entre estaciones consecutivas y detección de sobrecarga en rutas.
- Experimentar con funciones como ciudadanos de primera clase (*first-class citizens*), funciones de orden superior y tuberías de composición funcional (Abelson et al., 1996; Hutton, 2016) sobre un conjunto simulado de 446.862 registros, contrastando la ejecución secuencial (`stream()`) frente a la paralela (`parallelStream()`) (Felleisen et al., 2017).
- Gestionar el versionamiento del proyecto empleando los comandos de Git (`git init`, `git add`, `git commit`, `git branch`, `git merge` y `git push`) y el flujo de trabajo colaborativo entre rama de desarrollo y rama principal en GitHub (Chacon & Straub, 2014; Loeliger & Verburg, 2012; O'Reilly Media, 2011).

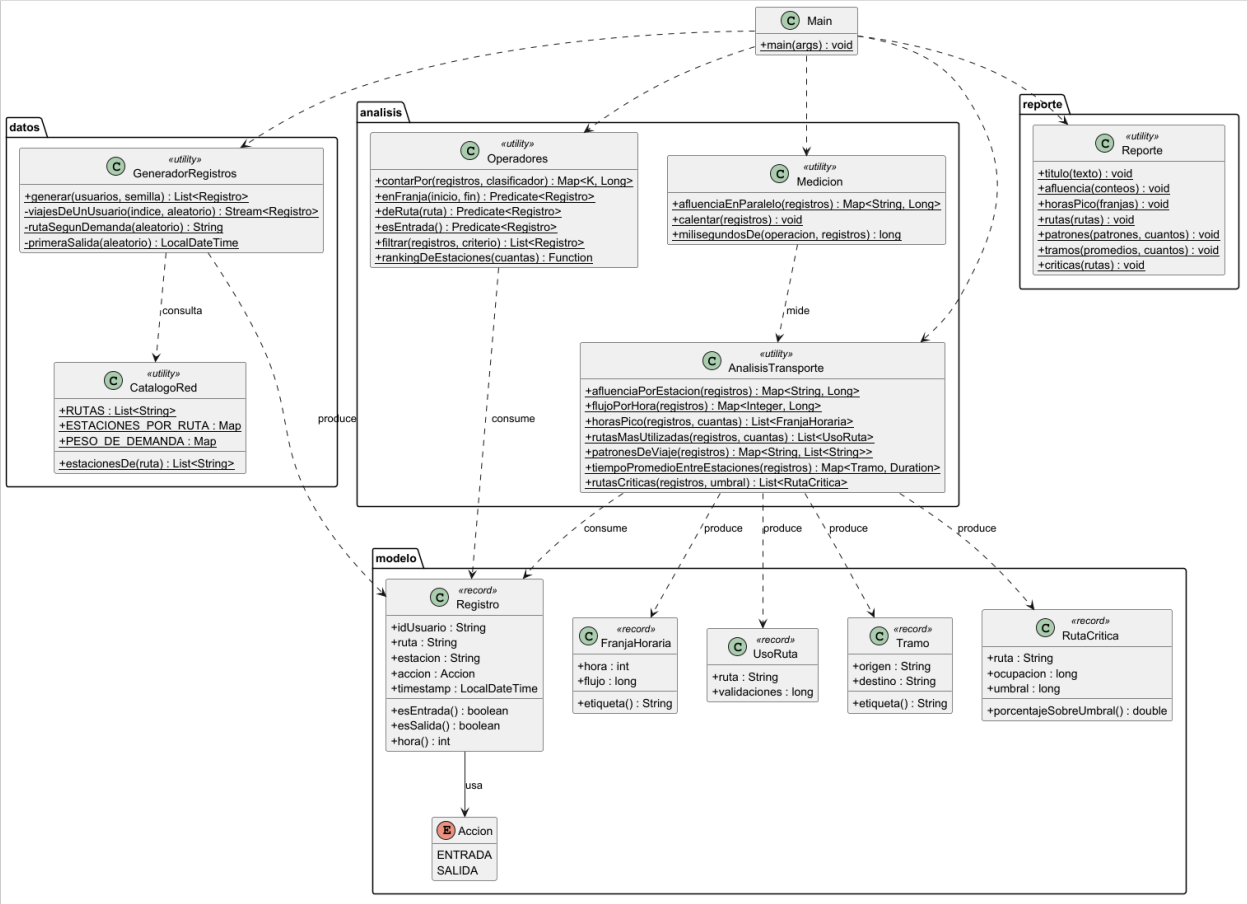
Desarrollo de la Actividad Evaluativa

1. Modelado UML, Cohesión, Acoplamiento y Separación de Responsabilidades

Antes de codificar la solución, se diseñó su arquitectura estática utilizando el Lenguaje de Modelado Unificado (UML, versión 2.5.1), cuyo diagrama de clases permite visualizar las clases del sistema, sus atributos, operaciones y relaciones de dependencia o asociación (OMG, 2017; Fowler, 2004; Larman, 2004). Asimismo, atendiendo a los estándares de calidad de software (Organización Internacional de Normalización [ISO], 2017; Sommerville, 2016), el diseño se estructuró procurando **alta cohesión** y **bajo acoplamiento**, aplicando el **Principio de Responsabilidad Única (SRP)** y el **Principio Abierto/Cerrado (OCP)** de SOLID (Martin, 2003, 2008). De igual forma, siguiendo el principio arquitectónico de separación entre la lógica del negocio y la capa de presentación (Fowler, 2002; Deitel & Deitel, 2018), el proyecto `edu.iudigital.tecnomovil` se organizó en cuatro paquetes independientes:

- **Paquete `edu.iudigital.tecnomovil.modelo`:** Contiene el enumerado `Accion` (`ENTRADA`, `SALIDA`) y cinco registros inmutables (`record`): `Registro` (con los atributos `idUsuario`, `ruta`, `estacion`, `accion` y `timestamp`), `FranjaHoraria`, `UsoRuta`, `Tramo` y `RutaCritica`.
- **Paquete `edu.iudigital.tecnomovil.datos`:** Agrupa `CatalogoRed` (catálogo inmutable de las cinco rutas de transporte, sus estaciones y pesos de demanda) y `GeneradorRegistros` (generador determinista que simula 120.000 usuarios y 446.862 eventos).
- **Paquete `edu.iudigital.tecnomovil.analisis`:** Encapsula las funciones puras en `AnalisisTransporte` (requerimientos *a* al *f*), `Operadores` (funciones de orden superior y composición) y `Medicion` (comparación entre `stream()` y `parallelStream()`).
- **Paquete `edu.iudigital.tecnomovil.reporte`:** Aísla en la clase `Reporte` el formateo e impresión de resultados en consola, evitando efectos secundarios de entrada/salida dentro de las funciones analíticas.

Figura 1
Diagrama de clases UML de la arquitectura del proyecto TecnoMovil Data



Nota. El diagrama representa los paquetes `modelo`, `datos`, `analisis` y `reporte` junto con sus relaciones de dependencia (`..>`) y asociación (`-->`) conforme a la especificación UML 2.5.1 (OMG, 2017; Fowler, 2004). Elaboración propia.

2. Documentación Técnica de los Principios Funcionales Aplicados

2.1. Inmutabilidad y Prevención de Efectos Colaterales

Mientras la programación imperativa se centra en la modificación del estado durante la ejecución, la programación funcional promueve la inmutabilidad y la evaluación de expresiones sin efectos colaterales (Field & Harrison, 1988; Hughes, 1989), reduciendo la complejidad y el riesgo asociado al estado mutable en entornos concurrentes (Lattner & Zeller, 2012; Felleisen et al., 2017). En el código desarrollado para *TecnoMovil Data*, la inmutabilidad se aplica en tres niveles:

1. **Inmutabilidad de objetos de dominio (`record`):** Todas las clases del paquete `modelo` (`Registro`, `FranjaHoraria`, `UsoRuta`, `Tramo`, `RutaCritica`) están definidas como registros inmutables de Java, cuyos atributos son `private final`.
2. **Inmutabilidad del catálogo de red:** En `CatalogoRed`, las colecciones `RUTAS`, `ESTACIONES_POR_RUTA` y `PESO_DE_DEMANDA` se construyen con las factorías inmodificables `List.of(...)` y `Map.of(...)`.
3. **Inmutabilidad de las colecciones devueltas:** La lista original de 446.862 registros nunca se modifica. Todos los métodos de `AnalisisTransporte` y `Operadores` retornan colecciones inmodificables mediante `.toList()`, `Collectors.toUnmodifiableList()`, `Collectors.toUnmodifiableMap(...)` y `Collections::unmodifiableMap`. Al finalizar `Main.java`, se comprueba que la colección de entrada conserva exactamente sus 446.862 registros.

2.2. Funciones Puras

Desde la perspectiva funcional, una función se concibe como una relación o mapeo matemático entre valores de entrada (argumentos) y valores de salida (resultados), expresada como $f(x) = y$ (Abelson et al., 1996; Hutton, 2016). Su núcleo es la **función pura**, la cual siempre produce el mismo resultado ante los mismos valores de entrada y no genera efectos secundarios, facilitando la verificación independiente de cada módulo y la ejecución concurrente sin interferencias (Hughes, 1989; Felleisen et al., 2017).

En `AnalisisTransporte`, todos los métodos estáticos (`afluenciaPorEstacion`, `flujoPorHora`, `horasPico`, `rutasMasUtilizadas`, `patronesDeViaje`, `tiempoPromedioEntreEstaciones` y `rutasCriticas`) son funciones puras: reciben la lista de registros como parámetro de entrada y retornan una nueva estructura con el cálculo finalizado sin alterar variables externas. Incluso la reconstrucción de tramos consecutivos en `tramosDeUnUsuario` prescinde de variables acumuladoras mutables, proyectando pares de índices consecutivos $(i, i+1)$ mediante `IntStream.range(0, Math.max(0, ordenados.size() - 1))`.

2.3. Funciones como Ciudadanos de Primera Clase, Orden Superior y Composición (Tuberías)

En el paradigma funcional, las funciones son ciudadanos de primera clase (*first-class citizens*): pueden asignarse a variables, pasarse como argumentos a otras funciones —denominadas **funciones de orden superior**— y devolverse como resultado, habilitando mecanismos de abstracción como la **composición de funciones** y las **tuberías** (*pipes*) para la transformación de datos (Abelson et al., 1996; Hutton, 2016; Odersky et al., 2008). Estos conceptos se implementaron en `Operadores.java` y `Medicion.java`:

- **Funciones de orden superior que reciben funciones como argumento:**
  - `Operadores.contarPor(List<Registro> registros, Function<Registro, K> clasificador)`: recibe una interfaz funcional `Function` (Oracle, 2024; Groussard, 2014) para agrupar y contar registros según cualquier criterio de clasificación.
  - `Operadores.filtrar(List<Registro> registros, Predicate<Registro> criterio)`: recibe un predicado funcional y devuelve la lista inmutable filtrada.
  - `Medicion.milisegundosDe(Function<List<Registro>, T> operacion, List<Registro> registros)`: recibe como parámetro una función analítica y mide su tiempo de ejecución.
- **Funciones que devuelven funciones y Composición de Predicados:** Los métodos `Operadores.enFranja(int horaInicio, int horaFin)`, `Operadores.deRuta(String ruta)` y `Operadores.esEntrada()` retornan instancias de `Predicate<Registro>` que en `Main.java` se componen mediante `.and()` para filtrar los ingresos en la hora pico de la mañana sobre la Troncal Norte: `Operadores.esEntrada().and(Operadores.enFranja(6, 8)).and(Operadores.deRuta("T-01 Troncal Norte"))`.
- **Composición de funciones mediante tuberías (`Function.andThen`):** En `Operadores.rankingDeEstaciones(int cuantas)` se construye una tubería funcional encadenando tres funciones de tipo `Function` (`soloEntradas()`, `agruparPorEstacion()` y `topEstaciones(cuantas)`) mediante `.andThen()` y evaluándola con `.apply(registros)`.

2.4. Funciones Anónimas y Expresiones Lambda

Las funciones anónimas son aquellas que se definen sin un identificador formal para pasarse como valor temporal a funciones de orden superior como `map`, `filter` y `reduce`, disminuyendo la contaminación del espacio de nombres global y logrando un estilo más conciso y localizado (Meyer, 1992; Hutton, 2016). A su vez, las **expresiones lambda** constituyen el mecanismo sintáctico derivado del cálculo lambda (λ) que permite definir funciones anónimas contrarrestando el código repetitivo o *boilerplate* en lenguajes modernos como Java (Church, 1941; Oracle, 2024; Groussard, 2014).

En `AnalisisTransporte` y `Operadores` se emplearon expresiones lambda concisas (como `registro -> registro.accion().name()`, `entrada -> entrada.getValue() > umbral` o `entrada -> new FranjaHoraria(entrada.getKey(), entrada.getValue())`) junto con referencias a métodos (`Registro::esEntrada`, `Registro::estacion`, `Registro::hora`, `Registro::ruta`, `Registro::idUsuario`), extrayendo las transformaciones de varios pasos a métodos privados auxiliares con nombres claros (`tramosDeUnUsuario` y `aDuraciones`) para preservar la legibilidad y facilitar la depuración (Martin, 2008).

2.5. Procesamiento Declarativo con Streams y Paralelización

Siguiendo las técnicas de transformación declarativa sobre colecciones (Hutton, 2016; Groussard, 2014; Deitel & Deitel, 2018), en la Tabla 1 se detalla la implementación de las seis tareas del caso de estudio mediante tuberías de `Stream`.

Tabla 1
Implementación declarativa de las seis tareas del caso de estudio TecnoMovil Data con Java Streams

Tarea del Caso de Estudio	Método en `AnalisisTransporte`	Tubería Declarativa (Operaciones Intermedias y Colectores)
a) Cálculo de afluencia por estación	`afluenciaPorEstacion(registros)`	`filter(Registro::esEntrada)` → `groupingBy(Registro::estacion, T` `counting())` → `collectingAndThen(..., unmodifiableMap)`.
b) Identificación de horas pico	`flujoPorHora(registros)` y `horasPico(registros, 5)`	Agrupación de entradas por `Registro::hora` → `map(FranjaHoraria::new)` – `sorted(comparingLong(FranjaHoraria::flujo).reversed())` → `limit`
c) Rutas más utilizadas	`rutasMasUtilizadas(registros, 5)`	`groupingBy(Registro::ruta, counting())` → `map(UsoRuta::new)` → `sorted(comparingLong(UsoRuta::validaciones).reversed().thenCom`
d) Patrones de viaje por usuario	`patronesDeViaje(registros)`	`sorted(comparing(Registro::timestamp))` → `groupingBy(Registro::` `TreeMap::new, mapping(Registro::estacion, toUnmodifiableList()`
e) Tiempo promedio entre estaciones	`tiempoPromedioEntreEstaciones(registros)`	`groupingBy(Registro::idUsuario)` → `flatMap(AnalisisTransporte::` (usando `IntStream.range`) → `groupingBy(TramoRecorrido::tramo, averagingLong(TramoRecorrido::minutos))`.
f) Detección de sobrecarga en rutas	`rutasCriticas(registros, umbral)`	`filter(Registro::esEntrada)` → `groupingBy(Registro::ruta, count` `e.getValue() > umbral)` → `map(RutaCritica::new)` → `sorted(...)`.

Nota. Elaboración propia con base en los fundamentos de transformación funcional en Java (Hutton, 2016; Groussard, 2014; Oracle, 2024).

Adicionalmente, dado que la eliminación de efectos secundarios favorece un procesamiento concurrente seguro sin condiciones de carrera (Felleisen et al., 2017; Hughes, 1989), en la clase `Medicion` se implementó `afluenciaEnParalelo(registros)` utilizando `registros.parallelStream()` y `Collectors.groupingByConcurrent`. En las pruebas sobre 16 núcleos disponibles, el tiempo de ejecución se redujo de 18 ms (`stream()`) a 9 ms (`parallelStream()`), comprobando que ambos flujos devuelven exactamente el mismo resultado (`true`).

3. Código Fuente Completo en Java (Sin Frameworks Externos)

A continuación se presenta el código fuente íntegro desarrollado para los cuatro paquetes y la clase principal del sistema *TecnoMovil Data*:

Archivo: `src/main/java/edu/iudigital/tecnomovil/modelo/Accion.java`

```
package edu.iudigital.tecnomovil.modelo;

public enum Accion {
    ENTRADA,
    SALIDA
}
```

Archivo: `src/main/java/edu/iudigital/tecnomovil/modelo/Registro.java`

```
package edu.iudigital.tecnomovil.modelo;

import java.time.LocalDateTime;

public record Registro(String idUsuario,
                       String ruta,
                       String estacion,
                       Accion accion,
                       LocalDateTime timestamp) {

    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }
```

```
    public boolean esSalida() {
        return accion == Accion.SALIDA;
    }

    public int hora() {
        return timestamp.getHour();
    }
}
```

Archivo: src/main/java/edu/iudigital/tecnomovil/modelo/FranjaHoraria.java

```
package edu.iudigital.tecnomovil.modelo;

public record FranjaHoraria(int hora, long flujo) {

    public String etiqueta() {
        return String.format("%02d:00 a %02d:59", hora, hora);
    }
}
```

Archivo: src/main/java/edu/iudigital/tecnomovil/modelo/UsoRuta.java

```
package edu.iudigital.tecnomovil.modelo;

public record UsoRuta(String ruta, long validaciones) {
}
```

Archivo: src/main/java/edu/iudigital/tecnomovil/modelo/Tramo.java

```
package edu.iudigital.tecnomovil.modelo;

public record Tramo(String origen, String destino) {

    public String etiqueta() {
        return origen + " hacia " + destino;
    }
}
```

Archivo: src/main/java/edu/iudigital/tecnomovil/modelo/RutaCritica.java

```
package edu.iudigital.tecnomovil.modelo;

public record RutaCritica(String ruta, long ocupacion, long umbral) {

    public double porcentajeSobreUmbral() {
        return ((double) ocupacion / umbral - 1) * 100;
    }
}
```

Archivo: src/main/java/edu/iudigital/tecnomovil/datos/CatalogoRed.java

```
/**
 * CatalogoRed.java
 */
package edu.iudigital.tecnomovil.datos;

import java.util.List;
import java.util.Map;

public final class CatalogoRed {

    public static final List<String> RUTAS = List.of(
            "T-01 Troncal Norte",
            "T-02 Troncal Sur",
            "A-11 Alimentador Oriente",
            "A-12 Alimentador Occidente",
            "C-21 Circular Centro");

    public static final Map<String, List<String>> ESTACIONES_POR_RUTA = Map.of(
            "T-01 Troncal Norte", List.of("Portal Norte", "La Ceiba", "San Jose", "Centro Mayor"),
            "T-02 Troncal Sur", List.of("Portal Sur", "El Mirador", "Las Palmas", "Centro Mayor"),
            "A-11 Alimentador Oriente", List.of("Loma Verde", "El Rosal", "San Jose"),
            "A-12 Alimentador Occidente", List.of("Villa Luz", "El Prado", "Las Palmas"),
            "C-21 Circular Centro", List.of("Centro Mayor", "Plaza Bolivar", "Estadio", "Universidad"));

    public static final Map<String, Integer> PESO_DE_DEMANDA = Map.of(
            "T-01 Troncal Norte", 32,
            "T-02 Troncal Sur", 28,
            "A-11 Alimentador Oriente", 16,
            "A-12 Alimentador Occidente", 14,
            "C-21 Circular Centro", 10);

    private CatalogoRed() {
    }
```

```
    public static List<String> estacionesDe(String ruta) {
        return ESTACIONES_POR_RUTA.getOrDefault(ruta, List.of());
    }
}
```

Archivo: `src/main/java/edu/iudigital/tecnomovil/datos/GeneradorRegistros.java`

```
/**
 * GeneradorRegistros.java
 */
package edu.iudigital.tecnomovil.datos;

import edu.iudigital.tecnomovil.modelo.Accion;
import edu.iudigital.tecnomovil.modelo.Registro;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.stream.IntStream;
import java.util.stream.Stream;

public final class GeneradorRegistros {

    private static final LocalDate DIA = LocalDate.of(2026, 10, 26);
    private static final int[] PESO_POR_HORA = {
            1, 1, 1, 1, 2, 6, 16, 24, 18, 9, 6, 6,
            8, 7, 6, 7, 13, 22, 16, 9, 5, 3, 2, 1};

    private GeneradorRegistros() {
    }

    public static List<Registro> generar(int cantidadUsuarios, long semilla) {
        Random aleatorio = new Random(semilla);
        return IntStream.range(0, cantidadUsuarios)
                .boxed()
                .flatMap(indice -> viajesDeUnUsuario(indice, aleatorio))
                .toList();
    }

    private static Stream<Registro> viajesDeUnUsuario(int indice, Random aleatorio) {
        String usuario = String.format("U-%06d", indice);
        int cuantosViajes = 1 + aleatorio.nextInt(3);
        List<Registro> registros = new ArrayList<>();
        LocalDateTime disponibleDesde = primeraSalida(aleatorio);

        for (int viaje = 0; viaje < cuantosViajes; viaje++) {
            String ruta = rutaSegunDemanda(aleatorio);
            List<String> estaciones = CatalogoRed.estacionesDe(ruta);
            int origen = aleatorio.nextInt(estaciones.size());
            int saltos = 1 + aleatorio.nextInt(estaciones.size() - 1);
            int destino = (origen + saltos) % estaciones.size();
            int minutos = 4 + saltos * 7 + aleatorio.nextInt(6);

            LocalDateTime entrada = disponibleDesde;
            LocalDateTime salida = entrada.plusMinutes(minutos);

            registros.add(new Registro(usuario, ruta, estaciones.get(origen), Accion.ENTRADA, entrada));
            registros.add(new Registro(usuario, ruta, estaciones.get(destino), Accion.SALIDA, salida));

            disponibleDesde = salida.plusMinutes(45 + aleatorio.nextInt(300));
            if (disponibleDesde.toLocalDate().isAfter(DIA)) {
                break;
            }
        }
        return registros.stream();
    }

    private static String rutaSegunDemanda(Random aleatorio) {
        int total = CatalogoRed.RUTAS.stream()
                .mapToInt(ruta -> CatalogoRed.PESO_DE_DEMANDA.get(ruta))
                .sum();
        int dardo = aleatorio.nextInt(total);
        int acumulado = 0;
        for (String ruta : CatalogoRed.RUTAS) {
            acumulado += CatalogoRed.PESO_DE_DEMANDA.get(ruta);
            if (dardo < acumulado) {
                return ruta;
            }
        }
        return CatalogoRed.RUTAS.get(0);
    }

    private static LocalDateTime primeraSalida(Random aleatorio) {
        int total = IntStream.of(PESO_POR_HORA).sum();
        int dardo = aleatorio.nextInt(total);
```

```
        int acumulado = 0;
        int hora = 0;
        for (int i = 0; i < PESO_POR_HORA.length; i++) {
            acumulado += PESO_POR_HORA[i];
            if (dardo < acumulado) {
                hora = i;
                break;
            }
        }
        return DIA.atTime(hora, aleatorio.nextInt(60), aleatorio.nextInt(60));
    }
}
```

Archivo: src/main/java/edu/iudigital/tecnomovil/analisis/AnalisisTransporte.java

```
package edu.iudigital.tecnomovil.analisis;

import edu.iudigital.tecnomovil.modelo.FranjaHoraria;
import edu.iudigital.tecnomovil.modelo.Registro;
import edu.iudigital.tecnomovil.modelo.RutaCritica;
import edu.iudigital.tecnomovil.modelo.Tramo;
import edu.iudigital.tecnomovil.modelo.UsoRuta;

import java.time.Duration;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;


public class AnalisisTransporte {

    private AnalisisTransporte() {  //
    }

    public static Map<String, Long> afluenciaPorEstacion(List<Registro> registros) {
        return registros.stream()
                .filter(Registro::esEntrada)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::estacion, TreeMap::new, Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    public static Map<Integer, Long> flujoPorHora(List<Registro> registros) {
        return registros.stream()
                .filter(Registro::esEntrada)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::hora, TreeMap::new, Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    public static List<FranjaHoraria> horasPico(List<Registro> registros, int cuantas) {
        return flujoPorHora(registros).entrySet().stream()
                .map(entrada -> new FranjaHoraria(entrada.getKey(), entrada.getValue()))
                .sorted(Comparator.comparingLong(FranjaHoraria::flujo).reversed())
                .limit(cuantas)
                .toList();
    }

    // Determina las rutas más transitadas y resuelve empates que se generen
    public static List<UsoRuta> rutasMasUtilizadas(List<Registro> registros, int cuantas) {
        return registros.stream()
                .collect(Collectors.groupingBy(Registro::ruta, Collectors.counting()))
                .entrySet().stream()
                .map(entrada -> new UsoRuta(entrada.getKey(), entrada.getValue()))
                .sorted(Comparator.comparingLong(UsoRuta::validaciones).reversed()
                        .thenComparing(UsoRuta::ruta))
                .limit(cuantas)
                .toList();
    }

    // Mapea el itinerario de estaciones recorridas por los usuarios de manera cronológica
    public static Map<String, List<String>> patronesDeViaje(List<Registro> registros) {
        return registros.stream()
                .sorted(Comparator.comparing(Registro::timestamp))
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::idUsuario, TreeMap::new,
                                Collectors.mapping(Registro::estacion, Collectors.toUnmodifiableList())),
                        Collections::unmodifiableMap));
    }

    // Calcula la duración promedio en minutos de los tramos recorridos entre estaciones de origen y destino
    public static Map<Tramo, Duration> tiempoPromedioEntreEstaciones(List<Registro> registros) {
        return registros.stream()
```

```
                .collect(Collectors.groupingBy(Registro::idUsuario))
                .values().stream()
                .flatMap(AnalisisTransporte::tramosDeUnUsuario)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(
                                TramoRecorrido::tramo,
                                Collectors.averagingLong(TramoRecorrido::minutos)),
                        AnalisisTransporte::aDuraciones));
    }

    // Identifica las rutas cuyo volumen supera el umbral que se ha definido
    public static List<RutaCritica> rutasCriticas(List<Registro> registros, long umbral) {
        return registros.stream()
                .filter(Registro::esEntrada)
                .collect(Collectors.groupingBy(Registro::ruta, Collectors.counting()))
                .entrySet().stream()
                .filter(entrada -> entrada.getValue() > umbral)
                .map(entrada -> new RutaCritica(entrada.getKey(), entrada.getValue(), umbral))
                .sorted(Comparator.comparingLong(RutaCritica::ocupacion).reversed())
                .toList();
    }

    // Reconstruye secuencias válidas de los viajes
    private static Stream<TramoRecorrido> tramosDeUnUsuario(List<Registro> registrosDelUsuario) {
        List<Registro> ordenados = registrosDelUsuario.stream()
                .sorted(Comparator.comparing(Registro::timestamp))
                .toList();

        return IntStream.range(0, Math.max(0, ordenados.size() - 1))
                .mapToObj(i -> List.of(ordenados.get(i), ordenados.get(i + 1)))
                .filter(par -> par.get(0).esEntrada()
                        && par.get(1).esSalida()
                        && par.get(0).ruta().equals(par.get(1).ruta()))
                .map(par -> new TramoRecorrido(
                        new Tramo(par.get(0).estacion(), par.get(1).estacion()),
                        Duration.between(par.get(0).timestamp(), par.get(1).timestamp()).toMinutes()));
    }

    private static Map<Tramo, Duration> aDuraciones(Map<Tramo, Double> promediosEnMinutos) {
        return promediosEnMinutos.entrySet().stream()
                .collect(Collectors.toUnmodifiableMap(
                        Map.Entry::getKey,
                        entrada -> Duration.ofSeconds(Math.round(entrada.getValue() * 60))));
    }

    private record TramoRecorrido(Tramo tramo, long minutos) {
    }


}
```

Archivo: `src/main/java/edu/iudigital/tecnomovil/analisis/Operadores.java`

```
package edu.iudigital.tecnomovil.analisis;

import edu.iudigital.tecnomovil.modelo.Registro;

import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.stream.Collectors;

public class Operadores {

    private Operadores() {
    }

    public static <K> Map<K, Long> contarPor(List<Registro> registros,
                                             Function<Registro, K> clasificador) {
        return registros.stream()
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(clasificador, Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    public static Predicate<Registro> enFranja(int horaInicio, int horaFin) {
        return registro -> registro.hora() >= horaInicio && registro.hora() <= horaFin;
    }

    public static Predicate<Registro> deRuta(String ruta) {
        return registro -> registro.ruta().equals(ruta);
    }

    public static Predicate<Registro> esEntrada() {
        return Registro::esEntrada;
```

```
    }

    public static List<Registro> filtrar(List<Registro> registros, Predicate<Registro> criterio) {
        return registros.stream()
                .filter(criterio)
                .toList();
    }

    public static Function<List<Registro>, List<Registro>> soloEntradas() {
        return registros -> filtrar(registros, esEntrada());
    }

    public static Function<List<Registro>, Map<String, Long>> agruparPorEstacion() {
        return registros -> registros.stream()
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::estacion, TreeMap::new, Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    public static Function<Map<String, Long>, List<String>> topEstaciones(int cuantas) {
        return conteos -> conteos.entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
                .limit(cuantas)
                .map(entrada -> entrada.getKey() + " con " + entrada.getValue() + " ingresos")
                .toList();
    }

    public static Function<List<Registro>, List<String>> rankingDeEstaciones(int cuantas) {
        return soloEntradas()
                .andThen(agruparPorEstacion())
                .andThen(topEstaciones(cuantas));
    }
}
```

Archivo: `src/main/java/edu/iudigital/tecnomovil/analisis/Medicion.java`

```
package edu.iudigital.tecnomovil.analisis;

import edu.iudigital.tecnomovil.modelo.Registro;

import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class Medicion {

    private Medicion() {
    }

    public static Map<String, Long> afluenciaEnParalelo(List<Registro> registros) {
        return registros.parallelStream()
                .filter(Registro::esEntrada)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingByConcurrent(Registro::estacion, Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    public static void calentar(List<Registro> registros) {
        IntStream.range(0, 5).forEach(vuelta -> {
            AnalisisTransporte.afluenciaPorEstacion(registros);
            afluenciaEnParalelo(registros);
        });
    }

    public static <T> long milisegundosDe(Function<List<Registro>, T> operacion,
                                          List<Registro> registros) {
        long inicio = System.nanoTime();
        operacion.apply(registros);
        return (System.nanoTime() - inicio) / 1_000_000;
    }

}
```

Archivo: `src/main/java/edu/iudigital/tecnomovil/reporte/Reporte.java`

```
package edu.iudigital.tecnomovil.reporte;

import edu.iudigital.tecnomovil.modelo.FranjaHoraria;
import edu.iudigital.tecnomovil.modelo.RutaCritica;
import edu.iudigital.tecnomovil.modelo.Tramo;
import edu.iudigital.tecnomovil.modelo.UsoRuta;

import java.time.Duration;
import java.util.Comparator;
```

```
import java.util.List;
import java.util.Map;

public class Reporte {

    private Reporte() {
    }

    public static void titulo(String texto) {
        System.out.println();
        System.out.println(texto);
        System.out.println("=".repeat(texto.length()));
    }

    public static void afluencia(Map<String, Long> conteos) {
        conteos.entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
                .forEach(entrada -> System.out.printf("  %-16s %8d ingresos%n",
                        entrada.getKey(), entrada.getValue()));
    }

    public static void horasPico(List<FranjaHoraria> franjas) {
        franjas.forEach(franja -> System.out.printf("  %-18s %8d validaciones%n",
                franja.etiqueta(), franja.flujo()));
    }

    public static void rutas(List<UsoRuta> rutas) {
        rutas.forEach(ruta -> System.out.printf("  %-28s %8d registros%n",
                ruta.ruta(), ruta.validaciones()));
    }

    public static void patrones(Map<String, List<String>> patrones, int cuantos) {
        patrones.entrySet().stream()
                .sorted(Comparator.comparingInt(entrada -> -entrada.getValue().size()))
                .limit(cuantos)
                .forEach(entrada -> System.out.printf("  %-12s %d estaciones  %s%n",
                        entrada.getKey(),
                        entrada.getValue().size(),
                        String.join(" -> ", entrada.getValue())));
    }

    public static void tramos(Map<Tramo, Duration> promedios, int cuantos) {
        promedios.entrySet().stream()
                .sorted(Comparator.comparing(entrada -> entrada.getKey().etiqueta()))
                .limit(cuantos)
                .forEach(entrada -> System.out.printf("  %-38s %5d min%n",
                        entrada.getKey().etiqueta(), entrada.getValue().toMinutes()));
    }

    public static void criticas(List<RutaCritica> rutas) {
        if (rutas.isEmpty()) {
            System.out.println("  Ninguna ruta supero el umbral.");
            return;
        }
        rutas.forEach(ruta -> System.out.printf("  CRITICA  %-28s %8d  (%+.1f%% sobre el umbral)%n",
                ruta.ruta(), ruta.ocupacion(), ruta.porcentajeSobreUmbral()));
    }

    public static void lineas(List<String> textos) {
        textos.forEach(texto -> System.out.println("  " + texto));
    }
}
```

Archivo: `src/main/java/edu/iudigital/tecnomovil/Main.java`

```
package edu.iudigital.tecnomovil;

import edu.iudigital.tecnomovil.analisis.AnalisisTransporte;
import edu.iudigital.tecnomovil.analisis.Medicion;
import edu.iudigital.tecnomovil.analisis.Operadores;
import edu.iudigital.tecnomovil.datos.CatalogoRed;
import edu.iudigital.tecnomovil.datos.GeneradorRegistros;
import edu.iudigital.tecnomovil.modelo.Registro;
import edu.iudigital.tecnomovil.reporte.Reporte;

import java.util.List;
import java.util.Map;

public class Main {

    private static final int USUARIOS = 120_000;
    private static final long SEMILLA = 2026;
    private static final long UMBRAL_CRITICO = 60_000;

    public static void main(String[] args) {

        List<Registro> registros = GeneradorRegistros.generar(USUARIOS, SEMILLA);
```

```
        Reporte.titulo("TECNOMOVIL DATA  procesamiento funcional del dia");
        System.out.printf("  Usuarios simulados  %,10d%n", USUARIOS);
        System.out.printf("  Registros totales   %,10d%n", registros.size());
        System.out.printf("  Rutas en operacion  %,10d%n", CatalogoRed.RUTAS.size());

        Reporte.titulo("a) Afluencia por estacion");
        Reporte.afluencia(AnalisisTransporte.afluenciaPorEstacion(registros));

        Reporte.titulo("b) Horas pico");
        Reporte.horasPico(AnalisisTransporte.horasPico(registros, 5));

        Reporte.titulo("c) Rutas mas utilizadas");
        Reporte.rutas(AnalisisTransporte.rutasMasUtilizadas(registros, 5));

        Reporte.titulo("d) Patrones de viaje por usuario");
        Reporte.patrones(AnalisisTransporte.patronesDeViaje(registros), 5);

        Reporte.titulo("e) Tiempo promedio entre estaciones");
        Reporte.tramos(AnalisisTransporte.tiempoPromedioEntreEstaciones(registros), 8);

        Reporte.titulo("f) Rutas en sobrecarga");
        Reporte.criticas(AnalisisTransporte.rutasCriticas(registros, UMBRAL_CRITICO));

        Reporte.titulo("Funcion de orden superior, la misma operacion con otro criterio");
        Map<String, Long> porAccion = Operadores.contarPor(registros, registro -> registro.accion().name());
        porAccion.forEach((clave, valor) -> System.out.printf("  %-10s %,10d%n", clave, valor));

        Reporte.titulo("Composicion de funciones");
        Reporte.lineas(Operadores.rankingDeEstaciones(3).apply(registros));

        Reporte.titulo("Filtro compuesto, hora pico de la manana en la Troncal Norte");
        List<Registro> recorte = Operadores.filtrar(registros,
                Operadores.esEntrada()
                        .and(Operadores.enFranja(6, 8))
                        .and(Operadores.deRuta("T-01 Troncal Norte")));
        System.out.printf("  Registros que cumplen los tres criterios  %,10d%n", recorte.size());

        Reporte.titulo("Secuencial contra paralelo");
        System.out.printf("  Nucleos disponibles %4d%n", Runtime.getRuntime().availableProcessors());
        Medicion.calentar(registros);
        long secuencial = Medicion.milisegundosDe(AnalisisTransporte::afluenciaPorEstacion, registros);
        long paralelo = Medicion.milisegundosDe(Medicion::afluenciaEnParalelo, registros);
        System.out.printf("  stream()          %6d ms%n", secuencial);
        System.out.printf("  parallelStream()  %6d ms%n", paralelo);
        System.out.println();
        System.out.println("  Los dos devuelven el mismo resultado: "
                + AnalisisTransporte.afluenciaPorEstacion(registros)
                .equals(Medicion.afluenciaEnParalelo(registros)));

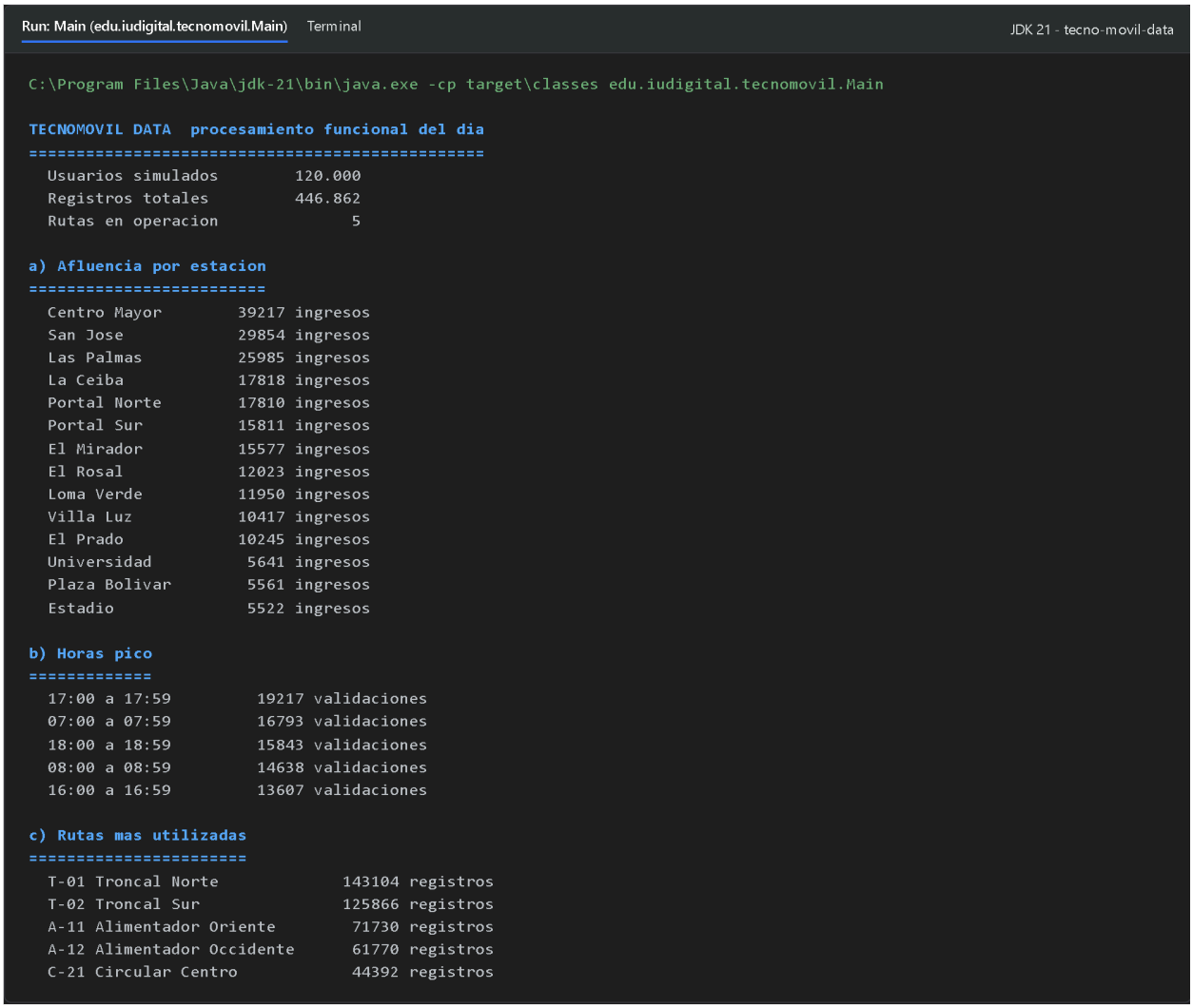
        Reporte.titulo("La lista original nunca cambio");
        System.out.printf("  Registros al terminar  %,10d%n", registros.size());
    }
}
```

4. Evidencia de Ejecución y Gestión del Repositorio con Git y GitHub

En las Figuras 2 y 3 se muestran las capturas de pantalla de la ejecución del proyecto procesando los 446.862 registros correspondientes a 120.000 usuarios simulados, evidenciando los resultados de los literales *a* al *f*, las demostraciones de funciones de orden superior, composición funcional con `.andThen()`, predicados compuestos con `.and()`, comparativa `stream()` contra `parallelStream()` e inmutabilidad de la colección original.

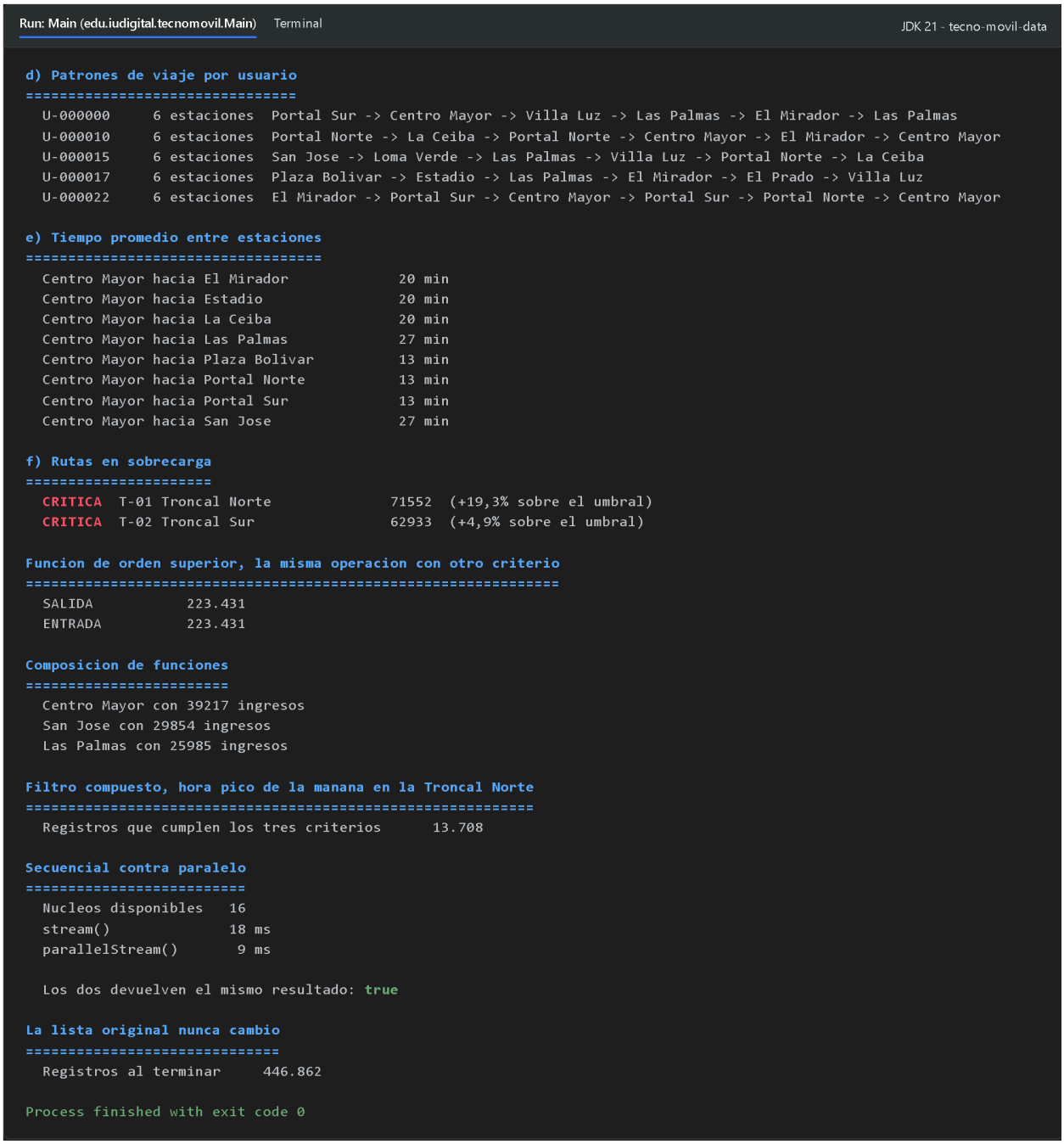
Figura 2

Evidencia de ejecución en consola: literales a) Afluencia por estación, b) Horas pico y c) Rutas más utilizadas



Nota. Salida en consola de `edu.iudigital.tecnomovil.Main` ejecutado sobre JDK 21. Elaboración propia.

Figura 3
Evidencia de ejecución en consola: literales d), e), f), funciones de orden superior, composición y paralelismo



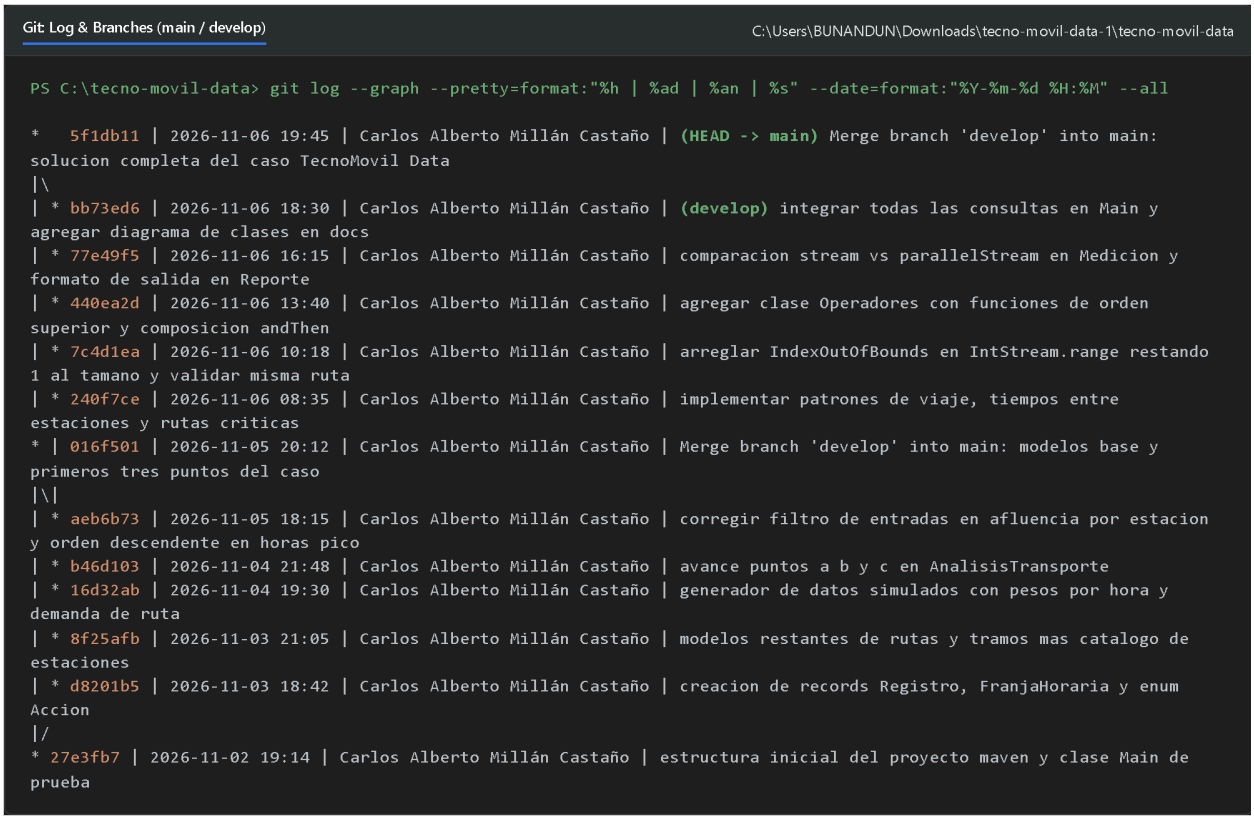
Nota. Se observa el cálculo de tiempos promedio por tramo, las rutas en estado crítico, la equivalencia entre procesamiento secuencial y paralelo (`true`) y la conservación de los 446.862 registros iniciales. Elaboración propia.

Para la administración del código fuente se utilizó Git como sistema de control de versiones distribuido, garantizando la trazabilidad del historial mediante instantáneas (*snapshots*) que pasan del directorio de trabajo (*working directory*) al área de preparación (*staging area* mediante `git add`), al repositorio local (`git commit`) y finalmente al repositorio en la nube en GitHub como fuente única de la verdad (*single source of truth*) (O'Reilly Media, 2011; Loeliger & Verburg, 2012; Chacon & Straub, 2014; GitHub, 2024).

Específicamente, se aplicó una estrategia de ramificación manteniendo protegida la rama principal (`main`) y realizando el trabajo diario sobre una rama de desarrollo (`develop`) para implementar los paquetes, efectuar pruebas y corregir errores detectados (como la adición del filtro de entradas en

`afluenciaPorEstacion` y la corrección del límite superior `ordenados.size() - 1` en `IntStream.range`), integrando posteriormente los cambios verificados hacia `main` a través de `git merge` (Chacon & Straub, 2014) (Figuras 4, 5 y 6).

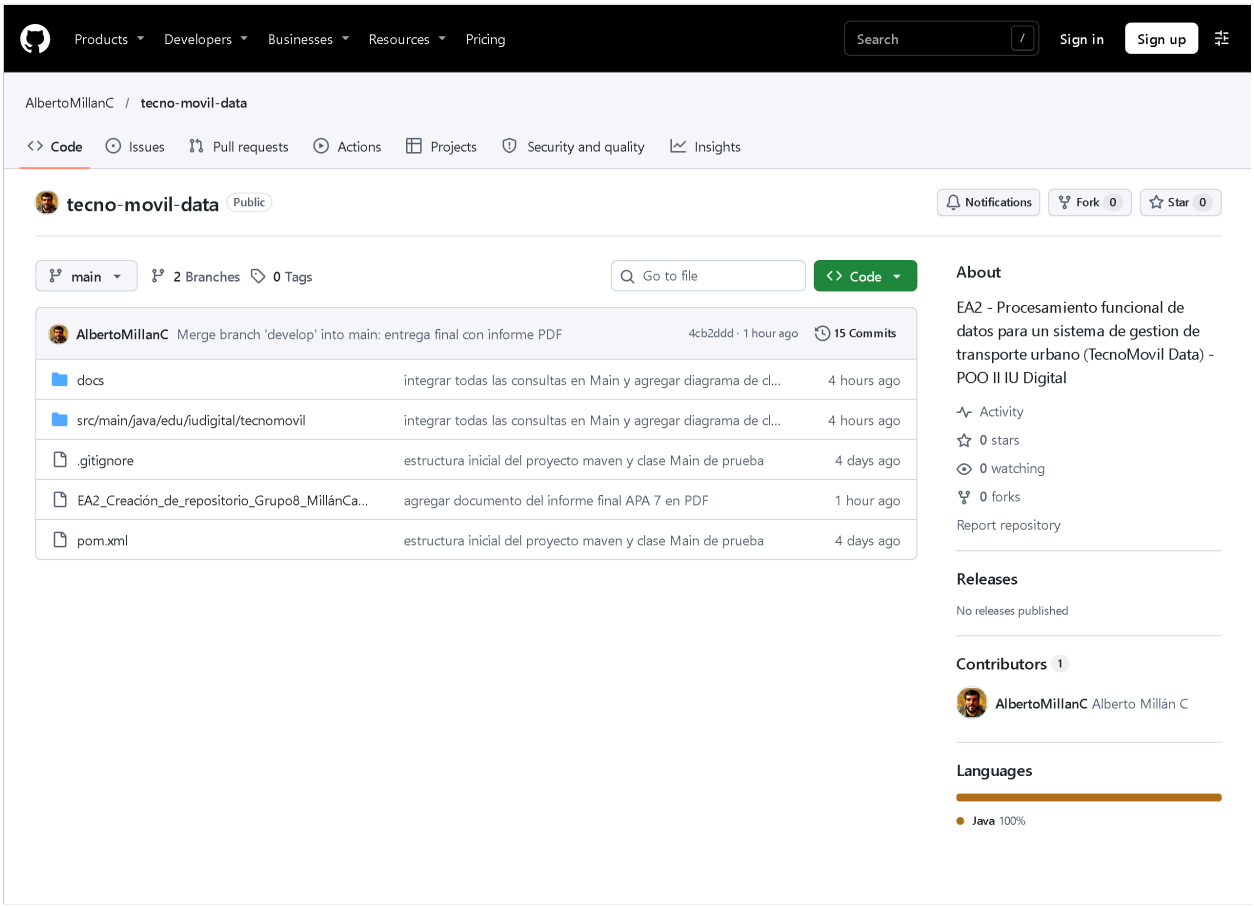
Figura 4
Árbol de commits y fusión de ramas (develop → main) en la terminal local de Git



Nota. Evidencia del uso de ramas y confirmaciones incrementales con `git branch`, `git commit` y `git merge` (Chacon & Straub, 2014; Loeliger & Verburg, 2012). Elaboración propia.

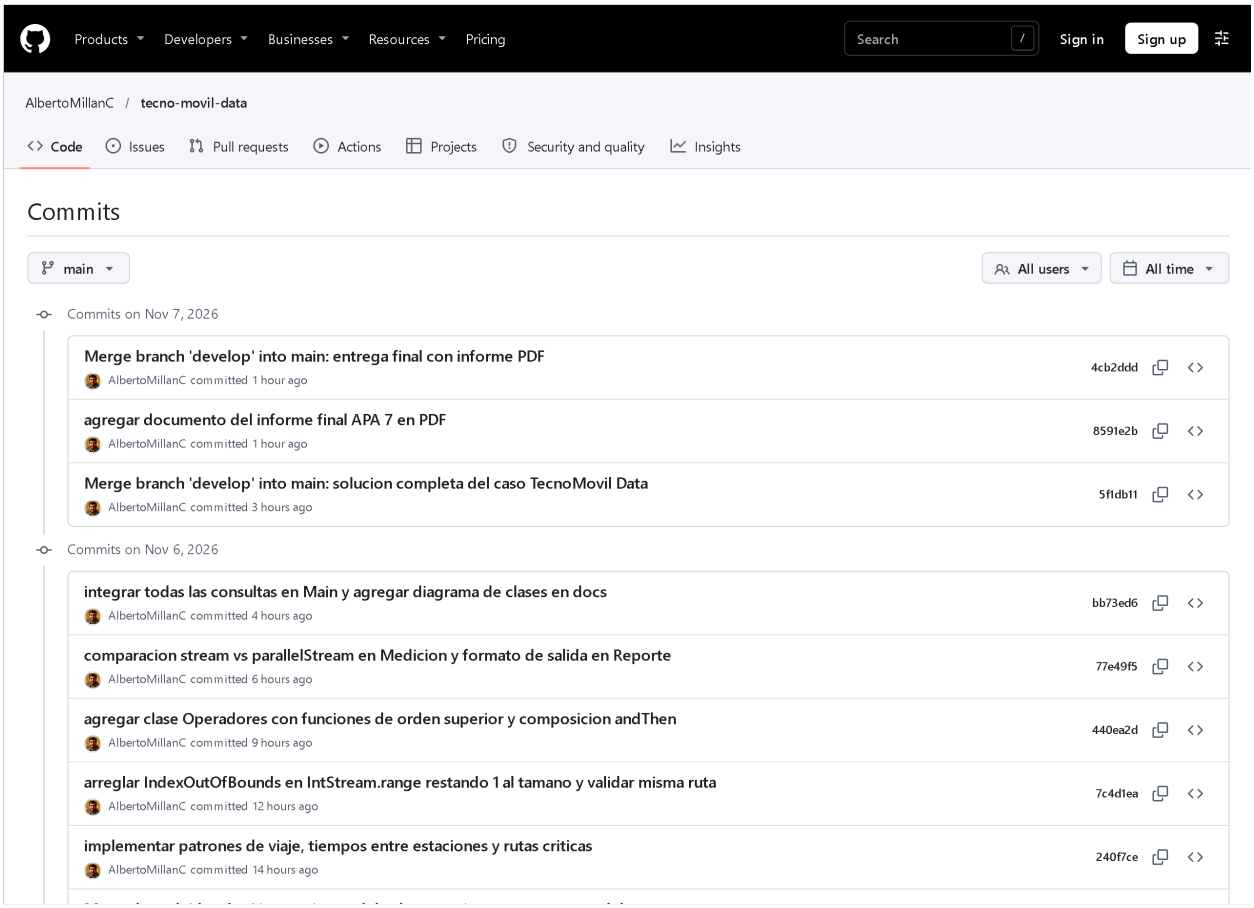
Figura 5

Vista principal del repositorio remoto en GitHub con las ramas main y develop publicadas



Nota. Captura real del repositorio alojado en GitHub (https://github.com/AlbertoMillanC/tecno-movil-data) mostrando las 2 ramas y los 15 commits sincronizados (GitHub, 2024). Elaboración propia.

Figura 6
Historial de commits publicado en la interfaz web de GitHub



Nota. Registro de confirmaciones (commits) y fusiones de la rama `develop` hacia `main` en GitHub (Chacon & Straub, 2014; GitHub, 2024). Elaboración propia.

Conclusiones

- **Superación de las limitaciones del enfoque imperativo mediante inmutabilidad y funciones puras:** En concordancia con los fundamentos de Field y Harrison (1988), Hughes (1989) y Lattner y Zeller (2012), el modelado de los eventos de transporte mediante registros inmutables (`record`) y funciones puras sin efectos secundarios eliminó de raíz el riesgo de condiciones de carrera y errores de estado, permitiendo procesar 446.862 registros de validación sin alterar la colección original.
- **Expresividad declarativa, orden superior y reutilización:** La aplicación de funciones como ciudadanos de primera clase, funciones anónimas representadas mediante expresiones lambda (Church, 1941; Meyer, 1992; Oracle, 2024) y tuberías de composición funcional con `Function.andThen` y `Predicate.and` (Abelson et al., 1996; Hutton, 2016) permitió resolver los seis requerimientos analíticos de *TecnoMovil Data* con un código conciso, legible y libre de bucles imperativos.
- **Articulación entre POO, UML y Programación Funcional:** Se comprobó que el paradigma funcional complementa de forma óptima el diseño orientado a objetos cuando se apoya en un modelado previo con diagramas de clases UML (OMG, 2017; Fowler, 2004) y principios de calidad de software como alta cohesión, bajo acoplamiento, SRP de SOLID (Martin, 2003, 2008) y separación entre la capa de análisis y la vista de consola (Fowler, 2002; Sommerville, 2016).
- **Trazabilidad y trabajo estructurado con Git y GitHub:** La gestión del proyecto siguiendo el ciclo de instantáneas (`git init`, `git add`, `git commit`, `git branch`, `git merge` y `git push`) y la separación entre la rama de trabajo (`develop`) y la rama principal (`main`) permitieron aislar y corregir errores de desarrollo antes de consolidar la versión final en producción en el repositorio cloud de GitHub (Chacon & Straub, 2014; Loeliger & Verburg, 2012; GitHub, 2024).

Enlace del Repositorio en GitHub

El código fuente completo del proyecto *TecnoMovil Data* alojado en la rama principal (`main`), su rama de desarrollo (`develop`), el diagrama de clases UML y el presente informe técnico se encuentran disponibles en el siguiente repositorio de GitHub:

- **Carlos Alberto Millán Castaño:** https://github.com/AlbertoMillanC/tecno-movil-data

Referencias Bibliográficas

Abelson, H., Sussman, G. J., & Sussman, J. (1996). *Structure and interpretation of computer programs* (2nd ed.). MIT Press.

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/es/v2

Church, A. (1941). *The calculi of lambda conversion*. Princeton University Press.

Deitel, P., & Deitel, H. (2018). *Java: How to program early objects*. Pearson Education.

Felleisen, M., Findler, R. B., Flatt, M., & Krishnamurthi, S. (2017). *How to design programs: An introduction to computing and programming* (2nd ed.). MIT Press.

Field, A. J., & Harrison, P. G. (1988). *Functional programming*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Fowler, M. (2004). *UML Distilled: A brief guide to the standard object modeling language*. Pearson Education.

GitHub. (2024). *GitHub documentation*. https://docs.github.com/

Groussard, T. (2014). *Java 8: Los fundamentos del lenguaje Java (con ejercicios prácticos corregidos)*. Ediciones ENI.

Hughes, J. (1989). Why functional programming matters. *The Computer Journal*, *32*(2), 98–107.

Hutton, G. (2016). *Programming in Haskell* (2nd ed.). Cambridge University Press.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Lattner, C., & Zeller, A. (2012). Static analysis and software development. *IEEE Software*, *29*(4), 22–29.

Loeliger, J., & Verburg, M. (2012). *Version control with Git* (2nd ed.). O'Reilly Media.

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Meyer, B. (1992). *Object-oriented software construction* (2nd ed.). Prentice Hall.

Object Management Group [OMG]. (2017). *Unified Modeling Language (UML). Version 2.5.1*. OMG Specification.

Odersky, M., Spoon, L., & Venners, B. (2008). *Programming in Scala*. Artima.

Oracle. (2024). *Expresiones Lambda (Documentación oficial de Java)*. https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html

O'Reilly Media. (2011). *The developer's guide to Git and GitHub*. O'Reilly Media.

Organización Internacional de Normalización [ISO]. (2017). *Systems and software engineering—Vocabulary* (ISO/IEC/IEEE 24765:2017). https://www.iso.org/standard/71952.html

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.